Recurrences

Outline

Introduction

Recurrences

The expansion method

The substitution method

The recursion-tree method

Towards the Master Theorem

Summary: which method when

Introduction

Introduction: from recursion to recurrences

- In the chapter *Recursion* we counted the steps of a recursive algorithm line by line
- The number of steps executed by a call depends on the number of steps executed by its recursive calls
- These are calls of the *same* algorithm on a *smaller* input
- The quantity we want to compute is therefore defined in terms of itself: this is a **recurrence**

The plan of the chapter

- First we define recurrences and show where they come from
- Then we study three methods to solve them, that is, to obtain their closed form or at least their order of growth:
  - the **expansion method**, which unrolls the recurrence until the boundary condition is reached;
  - the **substitution method**, which proves a guessed bound by induction;
  - the **recursion-tree method**, which visualises the costs of all the calls and sums them level by level.
- Finally we draw the recursion tree of a generic divide-and-conquer recurrence: the three behaviours that we observe there are the starting point of the Master Theorem, later in the course

Summary

Recurrences:

- the number of steps of a recursive algorithm is defined in terms of itself
- a recurrence describes it

Three methods to solve them:

- expansion, substitution, recursion tree

Recurrences

Introduction: Recurrences

- In this section we define recurrences, their boundary conditions and what it means to solve them
- We see how a recurrence arises from a recursive algorithm, and the most common shapes of recurrences
- We fix the technical details that can be neglected and the summations that all the methods of the chapter use

Definition 1: recurrence

*A **recurrence** is an equation (or an inequality) that describes a function $T : \mathbb{N} \to \mathbb{R}$ in terms of its values on smaller inputs.*

Boundary conditions and solutions

- Each recurrence has **boundary conditions**, which give the value of the function when the input size is small, that is, when n reaches (or is below) a specific value n_0. Precisely, the boundary conditions are

$$T(n) = g(n) \qquad \text{if } n \leq n_0,$$

  where $g(n)$ is the value of the function for the small values of n
- Typically $n_0 = 1$ and the boundary condition is $T(1) = \Theta(1)$
- **Solving a recurrence** means determining a closed form of $T(n)$ or, at least, its order of growth, that is, a function $h(n)$ such that $T(n) = \Theta(h(n))$ (or $T(n) = O(h(n))$, or $T(n) = \Omega(h(n))$)

Example (1/2)

Consider the recurrence

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n-1) + n & \text{if } n > 1 \end{cases}$$

The boundary condition is $T(1) = 1$ (here $n_0 = 1$); for $n > 1$ every value is obtained from the previous one:

- $T(1) = 1$ (boundary condition);
- $T(2) = T(1) + 2 = 1 + 2 = 3$;
- $T(3) = T(2) + 3 = 3 + 3 = 6$;
- $T(4) = T(3) + 4 = 6 + 4 = 10$.

Example (2/2)

- The recurrence defines $T(n)$ for every $n \in \mathbb{N}$, $n \geq 1$
- But computing $T(n)$ in this way requires $n - 1$ additions
- With the expansion method we will solve it and obtain the closed form

$$T(n) = \frac{n(n+1)}{2}$$

Recurrences and recursive algorithms

- Recurrences are the natural tool to determine the asymptotic complexity of **recursive algorithms**
- In this chapter recurrences are mathematical objects, and we use a generic function $T(n)$
- When we analyse a recursive algorithm, the function described by the recurrence is the number of steps $S(n)$ executed in the worst case on an input of size parameter n (see the chapter *Recursion*)
- The number of steps of a call is the number of steps executed by the call itself plus the number of steps executed by its recursive calls

Consider the following recursive algorithm
that computes the sum of the entries of an array.

Algorithm: RecursiveSum

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output: The sum $a[1] + a[2] + \cdots + a[n]$

```
1 function RecursiveSum(a, n)
2 |   if n = 1 then
3 |   |   return a[1]
4 |   end
5 |   s := RecursiveSum(a, n − 1)
6 |   return s + a[n]
7 end
```

Run in Colab: Python · C++ · Java

Example (1/2)

Let $S(n)$ be the number of steps executed by `RecursiveSum` on an array of size n. We count the executions of each line.

- If $n = 1$, lines 2 and 3 are executed once:

$$S(1) = 2$$

- If $n > 1$, lines 2, 5 and 6 are executed once (3 steps), and in addition the recursive call at line 5 executes $S(n-1)$ steps

Example (2/2)

Therefore

$$S(n) = \begin{cases} 2 & \text{if } n = 1 \\ S(n-1) + 3 & \text{if } n > 1 \end{cases}$$

- For instance, $S(2) = 2 + 3 = 5$, $S(3) = 5 + 3 = 8$ and $S(4) = 8 + 3 = 11$
- With the expansion method we will show that

$$S(n) = 3n - 1 = \Theta(n)$$

The shape of the recurrence mirrors the way the algorithm splits its input.

The most common shapes are the following.

Different forms of recurrences (1/2)

- **Divide-and-conquer.** A recursive algorithm divides the problem into a subproblems, each of size n/b, solves them recursively and combines their solutions:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n).$$

For example, the algorithm `MergeSort` gives rise to $T(n) = 2T(n/2) + \Theta(n)$

- **Unequal split.** A recursive algorithm might divide the problem into subproblems of different sizes. If the input is divided into a 1/3 and a 2/3 split:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + f(n),$$

where $f(n)$ is the time required to divide the problem and to combine the solutions of the subproblems

Different forms of recurrences (2/2)

- **Decrease-and-conquer.** Subproblems are not necessarily a constant fraction of the original problem. A recursive algorithm might create just one subproblem containing one element less than the original problem (as `RecursiveSum`):

$$T(n) = T(n-1) + f(n).$$

- **Several subproblems of size $n-1$.** The algorithm that solves the *Tower of Hanoi* puzzle with n disks moves $n-1$ disks, then one disk, then again $n-1$ disks:

$$T(n) = 2\,T(n-1) + 1,$$

whose solution is exponential in n (we will see it with the expansion method)

Definition 2: recurrence in standard form

*A recurrence is in **standard form** if it can be written as*

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

where

- n *is the size of the input (typically* $n \in \mathbb{N}$*);*
- $a \geq 1$ *is the number of subproblems to solve;*
- $\frac{n}{b}$ *is the size of each subproblem* $(b > 1)$*;*
- $f(n)$ *is the time required to divide the problem and to combine the solutions of the subproblems.*

Standard form with the boundary conditions

- Including the boundary conditions, a recurrence in standard form is

$$T(n) = \begin{cases} g(1) & \text{if } n = 1 \\ a\,T\left(\frac{n}{b}\right) + f(n) & \text{if } n > 1 \end{cases}$$

where $g(1)$ is the time required to solve a subproblem of size 1, typically $g(1) = \Theta(1)$

Recurrences given by inequalities

- Occasionally, we shall see recurrences that are not equalities but inequalities, such as

$$T(n) \leq a\,T\left(\frac{n}{b}\right) + f(n).$$

Such a recurrence states only an **upper bound** on $T(n)$: its solution is given in O-notation rather than in Θ-notation

- Similarly, if the inequality is reversed,

$$T(n) \geq a\,T\left(\frac{n}{b}\right) + f(n),$$

the recurrence gives only a **lower bound** on $T(n)$, and its solution is given in Ω-notation

Methods for solving recurrences

In this chapter we study three methods for solving recurrences:

1. The **expansion method** (also called iteration or unrolling): we repeatedly substitute the recurrence into itself until we reach the boundary condition, and we compute the resulting summation

2. The **substitution method**: we guess a bound and then use mathematical induction to prove that the guess is correct

3. The **recursion-tree method**: we convert the recurrence into a tree whose nodes represent the costs incurred at the various levels of the recursion, and we bound the resulting summations

Later in the course we study a fourth method, the **Master Theorem**, which provides directly the solution of recurrences in standard form.

Before applying the methods, we fix two technical points:

which details of a recurrence can be neglected,
and which summations appear most often.

Technicalities: floors and ceilings (1/2)

- We often neglect certain technical details when we state and solve recurrences
- Consider the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2\,T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- When n is odd, the two subproblems cannot both have size $n/2$ (which is not an integer): typically, they have sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$

Technicalities: floors and ceilings (2/2)

- The precise recurrence is therefore

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- The two recurrences have the same asymptotic solution
- Since we are interested in the order of growth, floors and ceilings can usually be omitted

Technicalities: exact powers

- For the same reason, when solving a recurrence in standard form we often assume that n is an **exact power of** b, that is,

$$n = b^k \qquad \text{for some } k \in \mathbb{N}$$

- In this way all the subproblem sizes $n/b, n/b^2, \ldots, 1$ are integers

- In the example above, we assume $n = 2^k$

Technicalities: boundary conditions

- Boundary conditions are another class of details that we typically ignore
- The running time of an algorithm on an input of constant size is a constant: the recurrences that arise from the analysis of algorithms generally satisfy

$$T(n) = \Theta(1) \qquad \text{for sufficiently small } n$$

- For convenience, we shall generally omit the boundary conditions and assume that $T(n)$ is constant for small n
- Changing the value of $T(1)$ typically changes the exact solution, but not its order of growth

Simplifications and rigour

- Omitting floors, ceilings and boundary conditions, and assuming that n is an exact power of b, simplifies the computations
- When a rigorous proof is required (for example in the substitution method), these details must be handled explicitly

All three methods end with a summation:
the costs of the single calls must be added up.

The following formulas are used throughout the chapter.

Observation 1: useful summations and logarithms (1/2)

- *The arithmetic series: for every* $n \geq 1$

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

- *The geometric series: for every real number* $x \neq 1$ *and every* $k \in \mathbb{N}$

$$\sum_{i=0}^{k} x^i = 1 + x + x^2 + \cdots + x^k = \frac{x^{k+1} - 1}{x - 1}.$$

In particular, for $x = 2$,

$$\sum_{i=0}^{k-1} 2^i = 2^k - 1.$$

Observation: useful summations and logarithms (2/2)

- *The infinite decreasing geometric series: for every real number x with $|x| < 1$*

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}.$$

- *Two properties of logarithms: for every $a, b, n > 0$ (with $b \neq 1$)*

$$a^{\log_b n} = n^{\log_b a}, \qquad \log_b n = \frac{\log_2 n}{\log_2 b}.$$

Proof (1/2)

Arithmetic series.

- We write the sum twice, once in increasing and once in decreasing order, and we add term by term
- We obtain n pairs, each with sum $n+1$: twice the sum is $n(n+1)$

Geometric series.

- Multiplying the sum by $x-1$, all the intermediate terms cancel:

$$(x-1)\sum_{i=0}^{k} x^i = \sum_{i=1}^{k+1} x^i - \sum_{i=0}^{k} x^i = x^{k+1} - 1,$$

  and we divide by $x-1 \neq 0$
- For $x=2$ and $k-1$ in place of k we obtain $2^k - 1$

Proof (2/2)

Infinite series.

- If $|x| < 1$, then $x^{k+1} \to 0$ for $k \to \infty$, and the finite sum tends to

$$\frac{0-1}{x-1} = \frac{1}{1-x}$$

Logarithms.

- We have

$$a^{\log_b n} = \left(b^{\log_b a}\right)^{\log_b n} = \left(b^{\log_b n}\right)^{\log_b a} = n^{\log_b a}$$

- Moreover $n = b^{\log_b n}$ gives $\log_2 n = \log_b n \cdot \log_2 b$



Two consequences

- Since all the terms are positive when $0 < x < 1$, every finite sum $\sum_{i=0}^{k} x^i$ is smaller than $\frac{1}{1-x}$

  The infinite series gives an **upper bound** on a finite geometric sum that does not depend on k

- The second property of the logarithms shows that logarithms in different bases differ by a constant factor

  Hence, inside Θ, O and Ω the base of the logarithm is irrelevant

Summary (1/2)

Recurrences:

- a function $T(n)$ described in terms of its values on smaller inputs
- boundary conditions give $T(n)$ for $n \leq n_0$, typically $T(1) = \Theta(1)$
- solving a recurrence: closed form or order of growth

Recursive algorithms:

- the number of steps $S(n)$ satisfies a recurrence, e.g. $S(n) = S(n-1) + 3$ for `RecursiveSum`
- standard form $T(n) = a\,T(n/b) + f(n)$, with $a \geq 1$ and $b > 1$
- other shapes: $T(n-1) + f(n)$, $2\,T(n-1) + 1$, $T(n/3) + T(2n/3) + f(n)$

Summary (2/2)

Technicalities:

- floors, ceilings and boundary conditions can usually be omitted
- we often assume that n is an exact power of b
- a rigorous proof must handle them explicitly

Useful summations:

- arithmetic series, finite and infinite geometric series
- $a^{\log_b n} = n^{\log_b a}$; the base of the logarithm is irrelevant inside Θ

The expansion method

Introduction: The expansion method

- The most direct way to solve a recurrence is to apply it again and again
- Each application replaces a value of T with a value on a smaller argument, and after enough applications only the boundary condition remains
- We solve five recurrences in this way, from linear to exponential running times

The expansion method

In the **expansion method** (also called **iteration** or **unrolling**) we:

1. substitute the recurrence into itself a few times, until we recognise the general pattern after k substitutions;
2. choose k such that the argument reaches the boundary condition;
3. compute (or bound) the resulting summation.

- The method gives an exact closed form when the summation can be computed
- The pattern recognised in step 1 can be proved rigorously by induction on k

We start with the recurrences of the decrease-and-conquer type,

$$T(n-1)+f(n),$$

and then move to the divide-and-conquer type,

$$a\,T(n/2)+f(n).$$

Example: $T(n) = T(n-1) + c$ (1/2)

Let $c, d > 0$ be constants and

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ T(n-1) + c & \text{if } n > 1 \end{cases}$$

Step 1 (expand).

$$\begin{aligned} T(n) &= T(n-1) + c \\ &= \big(T(n-2) + c\big) + c = T(n-2) + 2c \\ &= \big(T(n-3) + c\big) + 2c = T(n-3) + 3c \\ &\vdots \\ &= T(n-k) + k\,c. \end{aligned}$$

Example: $T(n) = T(n-1) + c$ (2/2)

Step 2 (boundary condition).

- The boundary condition is reached when $n - k = 1$, that is, $k = n - 1$

Step 3 (sum).

$$T(n) = T(1) + (n-1)\,c = c\,n + (d - c) = \Theta(n).$$

- For `RecursiveSum` we have $c = 3$ and $d = 2$, hence

$$S(n) = 3(n-1) + 2 = 3n - 1 = \Theta(n)$$

- This gives $S(2) = 5$, $S(3) = 8$, $S(4) = 11$, as computed directly from the recurrence

Example: $T(n) = T(n-1) + n$ (1/3)

Let

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n-1) + n & \text{if } n > 1 \end{cases}$$

Step 1 (expand).

$$\begin{aligned} T(n) &= T(n-1) + n \\ &= T(n-2) + (n-1) + n \\ &= T(n-3) + (n-2) + (n-1) + n \\ &\vdots \\ &= T(n-k) + (n-k+1) + \cdots + (n-1) + n. \end{aligned}$$

Example: $T(n) = T(n-1) + n$ (2/3)

Step 2 (boundary condition).

- $n - k = 1$, that is, $k = n - 1$

Step 3 (sum).

- Using the arithmetic series,

$$\begin{aligned} T(n) &= T(1) + 2 + 3 + \cdots + n \\ &= 1 + 2 + \cdots + n \\ &= \frac{n(n+1)}{2} = \Theta(n^2) \end{aligned}$$

Example: $T(n) = T(n-1) + n$ (3/3)

- Indeed,

$$\lim_{n\to\infty} \frac{n(n+1)/2}{n^2} = \lim_{n\to\infty} \frac{1}{2}\left(1 + \frac{1}{n}\right) = \frac{1}{2} \in \mathbb{R}_{>0}$$

- The closed form gives $T(1) = 1$, $T(2) = 3$, $T(3) = 6$, $T(4) = 10$, the values computed at the beginning of the chapter

Example: $T(n) = 2T(n-1) + 1$, the Tower of Hanoi (1/3)

Let

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\,T(n-1) + 1 & \text{if } n > 1 \end{cases}$$

Step 1 (expand).

$$\begin{aligned} T(n) &= 2\,T(n-1) + 1 \\ &= 2\big(2\,T(n-2) + 1\big) + 1 = 2^2\,T(n-2) + 2 + 1 \\ &= 2^2\big(2\,T(n-3) + 1\big) + 2 + 1 = 2^3\,T(n-3) + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^k\,T(n-k) + \sum_{i=0}^{k-1} 2^i. \end{aligned}$$

Example: $T(n) = 2T(n-1) + 1$, the Tower of Hanoi (2/3)

Step 2 (boundary condition).

- $n - k = 1$, that is, $k = n - 1$

Step 3 (sum).

- Using $\sum_{i=0}^{k-1} 2^i = 2^k - 1$,

$$\begin{aligned} T(n) &= 2^{n-1}\, T(1) + \left(2^{n-1} - 1\right) \\ &= 2^{n-1} + 2^{n-1} - 1 \\ &= 2^n - 1 = \Theta(2^n) \end{aligned}$$

Example: $T(n) = 2T(n-1) + 1$, the Tower of Hanoi (3/3)

- This is the number of moves needed to solve the Tower of Hanoi puzzle with n disks: the running time is **exponential** in n
- The function `hanoi` of the code of the course counts its moves; running it we obtain

n	1	2	3	4	5	6	7	8	10
moves	1	3	7	15	31	63	127	255	1023

that is, exactly $2^n - 1$ moves

The next two recurrences halve the size of the problem at each call.

To avoid floors we assume that n is an exact power of 2.

Example: $T(n) = T(n/2) + c$ (1/2)

Let $c, d > 0$ be constants, let $n = 2^k$ be an exact power of 2, and

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ T\left(\frac{n}{2}\right) + c & \text{if } n > 1 \end{cases}$$

Step 1 (expand).

$$T(n) = T\left(\frac{n}{2}\right) + c = T\left(\frac{n}{2^2}\right) + 2c = T\left(\frac{n}{2^3}\right) + 3c$$
$$= \cdots = T\left(\frac{n}{2^j}\right) + j\,c.$$

Example: $T(n) = T(n/2) + c$ (2/2)

Step 2 (boundary condition).

- $\frac{n}{2^j} = 1$, that is, $j = \log_2 n$

Step 3 (sum).

$$T(n) = T(1) + c \log_2 n = c \log_2 n + d = \Theta(\log_2 n).$$

- This recurrence describes, for instance, the recursive version of `BinarySearch`: at each call the size of the sub-array is halved

Example: $T(n) = 2T(n/2) + n$ (1/3)

Let $n = 2^k$ be an exact power of 2 and

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\,T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

We expand it as in the previous examples.

Example: $T(n) = 2T(n/2) + n$ (2/3)

Step 1 (expand).

$$
\begin{aligned}
T(n) &= 2\,T\left(\frac{n}{2}\right) + n \\
&= 2\left(2\,T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right) + n \\
&= 2^2\,T\left(\frac{n}{2^2}\right) + 2n \\
&= 2^2\left(2\,T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right) + 2n \\
&= 2^3\,T\left(\frac{n}{2^3}\right) + 3n \\
&\vdots \\
&= 2^j\,T\left(\frac{n}{2^j}\right) + j\,n.
\end{aligned}
$$

Example: $T(n) = 2T(n/2) + n$ (3/3)

Step 2 (boundary condition).

- $j = \log_2 n$, so that $2^j = n$

Step 3 (sum).

$$T(n) = n\,T(1) + n\log_2 n = n\log_2 n + n = \Theta(n\log_2 n).$$

Check:

- from the recurrence $T(2) = 2 + 2 = 4$, $T(4) = 8 + 4 = 12$, $T(8) = 24 + 8 = 32$
- from the closed form $T(8) = 8 \cdot 3 + 8 = 32$

When the expansion method works well

- The expansion method works well when the recurrence has a **single recursive term**
- With several recursive terms of different sizes (for example $T(n/3) + T(2n/3)$) the expansion quickly becomes unmanageable
- In these cases the recursion-tree method is more convenient

Summary

The expansion method:

- expand, reach the boundary condition, compute the summation
- it gives the exact closed form when the summation can be computed

Solutions found:

- $T(n-1)+c$: $\Theta(n)$ $\quad$ $T(n-1)+n$: $\frac{n(n+1)}{2} = \Theta(n^2)$
- $2T(n-1)+1$: $2^n - 1 = \Theta(2^n)$ (Tower of Hanoi)
- $T(n/2)+c$: $\Theta(\log_2 n)$ $\quad$ $2T(n/2)+n$: $n \log_2 n + n = \Theta(n \log_2 n)$

Limit: a single recursive term

The substitution method

Introduction: The substitution method

- The expansion method needs exact subproblem sizes: with floors and ceilings the pattern is no longer clean
- When we already have a good idea of the solution, a more robust approach is to *prove* it by induction
- We prove an upper and a lower bound for a recurrence with floors, we see how to strengthen a guess, and how a change of variables helps

The substitution method

The **substitution method** comprises two steps:

1. **guess** the form of the solution (for example $T(n) \leq c\, n \log_2 n$, with c an unknown constant);
2. use **mathematical induction** to find the constants and show that the solution works.

Remarks on the substitution method

- The name comes from the fact that, in the inductive step, we *substitute* the guessed bound for the values of the function on the smaller inputs
- The method is very powerful, but it requires a good guess. The guess can come from experience (similar recurrences), from the expansion method or from a recursion tree
- We can use the substitution method to prove upper bounds (O) and lower bounds (Ω) separately. Both together give Θ
- Since the asymptotic notation requires the bound only for $n \geq n_0$, we are free to choose n_0: this is useful when the bound does not hold for the smallest values of n

An example with floors: upper bound

- Consider the recurrence

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + n & \text{if } n > 1 \end{cases}$$

- With the expansion method (where n is a power of 2) we found $T(n) = n \log_2 n + n$, so we guess $T(n) = O(n \log_2 n)$
- We want to prove that there exist constants $c > 0$ and n_0 such that

$$T(n) \leq c\, n \log_2 n \qquad \text{for all } n \geq n_0$$

Upper bound: inductive step (1/2)

- Assume that the bound holds for all the values m with $n_0 \leq m < n$
- In particular for $m = \lfloor n/2 \rfloor$ (this requires $\lfloor n/2 \rfloor \geq n_0$):

$$T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor$$

Upper bound: inductive step (2/2)

Substituting into the recurrence:

$$
\begin{aligned}
T(n) &\leq 2\,c\,\lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \\
&\leq 2\,c\,\frac{n}{2}\log_2 \frac{n}{2} + n && \text{(since } \lfloor n/2 \rfloor \leq n/2\text{)} \\
&= c\,n\log_2 n - c\,n\log_2 2 + n \\
&= c\,n\log_2 n - c\,n + n \\
&\leq c\,n\log_2 n,
\end{aligned}
$$

where the last step holds as long as $c \geq 1$.

Upper bound: base cases (1/2)

- The induction needs the bound to hold for the boundary conditions. However, for $n = 1$:

$$T(1) = 1 > c \cdot 1 \cdot \log_2 1 = 0,$$

  for every c. So the bound *does not hold* for $n = 1$

- We take advantage of the freedom of choosing n_0: we prove the bound for $n \geq n_0 = 2$
- For $n \geq 4$ we have $\lfloor n/2 \rfloor \geq 2$, so the inductive step only uses values of T on arguments ≥ 2
- For $n = 2$ and $n = 3$ we have $\lfloor n/2 \rfloor = 1$, so the inductive step would use $T(1)$, which does not satisfy the bound. Therefore $n = 2$ and $n = 3$ must be treated as **base cases**

Upper bound: base cases (2/2)

- From the recurrence,

$$T(2) = 2\,T(1) + 2 = 4 \qquad \text{and} \qquad T(3) = 2\,T(1) + 3 = 5$$

- We need

$$4 \leq c \cdot 2 \log_2 2 = 2c \qquad \text{and} \qquad 5 \leq c \cdot 3 \log_2 3$$

- Both hold for $c \geq 2$ (since $3 \log_2 3 \approx 4.75$, we have $2 \cdot 3 \log_2 3 \approx 9.51 \geq 5$)

Observation 2: upper bound for $2T(\lfloor n/2 \rfloor) + n$

The function

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + n & \text{if } n > 1 \end{cases}$$

satisfies $T(n) \leq 2\,n \log_2 n$ *for all* $n \geq 2$. *Hence* $T(n) = O(n \log_2 n)$.

Proof

Let $c = 2$. We proceed by strong induction on $n \geq 2$.

Base cases $n = 2, 3$: $T(2) = 4 \leq 2 \cdot 2 \cdot \log_2 2 = 4$ and $T(3) = 5 \leq 2 \cdot 3 \log_2 3 \approx 9.51$.

Inductive step $n \geq 4$: we have $2 \leq \lfloor n/2 \rfloor < n$, so by the inductive hypothesis $T(\lfloor n/2 \rfloor) \leq 2\lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor$. Then

$$\begin{aligned} T(n) &\leq 4\lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \\ &\leq 2n \log_2 \frac{n}{2} + n \\ &= 2n \log_2 n - 2n + n \\ &\leq 2n \log_2 n. \end{aligned}$$

□

An example with floors: lower bound

- We now prove $T(n) = \Omega(n \log_2 n)$ for the same recurrence. We look for a constant $d > 0$ such that $T(n) \geq d\, n \log_2 n$ for all $n \geq 1$
- For the lower bound, the floor works against us: we need lower bounds on $\lfloor n/2 \rfloor$. For every $n \geq 2$:

$$2\lfloor n/2 \rfloor \geq n - 1 \qquad \text{and} \qquad \lfloor n/2 \rfloor \geq \frac{n}{3}$$

- The first holds because $\lfloor n/2 \rfloor \geq (n-1)/2$
- The second holds for $n = 2$ ($1 \geq 2/3$) and, for $n \geq 3$, because $(n-1)/2 \geq n/3$ is equivalent to $n \geq 3$

Lower bound: inductive step (1/2)

- Let $n \geq 2$ and assume $T(\lfloor n/2 \rfloor) \geq d \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor$
- Since $\lfloor n/2 \rfloor \geq 1$, the factor $\log_2 \lfloor n/2 \rfloor$ is non-negative, and

$$
\begin{aligned}
T(n) &\geq 2\, d\, \lfloor n/2 \rfloor \log_2 \lfloor n/2 \rfloor + n \\
&\geq d\,(n-1) \log_2 \frac{n}{3} + n \\
&= d\, n \log_2 n - d\, n \log_2 3 - d \log_2 n + d \log_2 3 + n \\
&\geq d\, n \log_2 n + \Big(n - d\, n \log_2 3 - d \log_2 n \Big).
\end{aligned}
$$

Lower bound: inductive step (2/2)

- Since $\log_2 n \leq n$, the term in parentheses is at least

$$n\,(1 - d\log_2 3 - d),$$

which is non-negative if

$$d \leq \frac{1}{1 + \log_2 3} \approx 0.387$$

- We choose $d = \frac{1}{4}$

Base case ($n = 1$):

$$T(1) = 1 \geq \frac{1}{4} \cdot 1 \cdot \log_2 1 = 0. \checkmark$$

Observation 3: lower bound for $2T(\lfloor n/2 \rfloor) + n$

The function

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + n & \text{if } n > 1 \end{cases}$$

satisfies $T(n) \geq \frac{1}{4}\, n \log_2 n$ *for all* $n \geq 1$. *Hence* $T(n) = \Omega(n \log_2 n)$.

Proof

By strong induction on $n \geq 1$, with $d = \frac{1}{4}$.

- For $n = 1$, $T(1) = 1 \geq 0$
- For $n \geq 2$ we have $1 \leq \lfloor n/2 \rfloor < n$, so the inductive hypothesis holds for $\lfloor n/2 \rfloor$, and the computation of the inductive step above gives

$$\begin{aligned} T(n) &\geq \tfrac{1}{4}\, n \log_2 n + n \left(1 - \tfrac{1}{4} \log_2 3 - \tfrac{1}{4}\right) \\ &\geq \tfrac{1}{4}\, n \log_2 n, \end{aligned}$$

since $1 - \frac{1}{4}(1 + \log_2 3) \approx 0.35 > 0$

□

The solution of $2T(\lfloor n/2 \rfloor) + n$

- From the two Observations above, the solution of the recurrence is
$$T(n) = \Theta(n \log_2 n)$$
- The floor does not change the order of growth found for n power of 2
- Numerically, for every n with $2 \leq n < 200\,000$ the ratio
$$\frac{T(n)}{n \log_2 n}$$
lies between 0.83 and 2

Sometimes the guess is correct,
but the induction does not work
because the inductive hypothesis is *not strong enough*.

When the guess must be strengthened (1/2)

- Consider

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + 1 & \text{if } n > 1 \end{cases}$$

- For n power of 2 one easily finds $T(n) = 2n - 1$ (for instance $T(2) = 3,\ T(4) = 7,\ T(8) = 15$)
- So we guess $T(n) = O(n)$, that is, $T(n) \leq c\,n$

When the guess must be strengthened (2/2)

- **Naive attempt.** Substituting the guess,

$$T(n) \leq 2\,c\,\lfloor n/2 \rfloor + 1 \leq c\,n + 1,$$

which does not imply $T(n) \leq c\,n$ for any choice of c

The induction fails, by a lower-order term (the constant 1)

- **Strengthened guess.** It is tempting to try a larger guess, such as $c\,n^2$, but this is not needed

Instead, we *subtract* a lower-order term: we guess

$$T(n) \leq c\,n - d,$$

with $d \geq 0$ a constant

The strengthened guess works

- Substituting the strengthened guess,

$$T(n) \leq 2\big(c\,\lfloor n/2 \rfloor - d\big) + 1 \leq c\,n - 2d + 1 \leq c\,n - d,$$

  where the last step holds as long as $d \geq 1$

- **Base case.** $T(1) = 1 \leq c - d$: with $d = 1$ we need $c \geq 2$

Observation 4: strengthened guess for $2T(\lfloor n/2 \rfloor) + 1$

The function

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2\,T\left(\lfloor n/2 \rfloor\right) + 1 & \text{if } n > 1 \end{cases}$$

satisfies $n \leq T(n) \leq 2n - 1$ *for all* $n \geq 1$. *Hence* $T(n) = \Theta(n)$.

Proof

By strong induction on n.

- For $n = 1$: $1 \leq T(1) = 1 \leq 2 \cdot 1 - 1$
- For $n \geq 2$, we have $1 \leq \lfloor n/2 \rfloor < n$ and, by the inductive hypothesis,

$$\lfloor n/2 \rfloor \leq T(\lfloor n/2 \rfloor) \leq 2\lfloor n/2 \rfloor - 1$$

- *Upper bound*:
  $T(n) \leq 2\big(2\lfloor n/2 \rfloor - 1\big) + 1 = 4\lfloor n/2 \rfloor - 1 \leq 2n - 1$
- *Lower bound*: $T(n) \geq 2\lfloor n/2 \rfloor + 1 \geq (n-1) + 1 = n$

□

Subtract a lower-order term

- When the induction fails only because of a **lower-order term**, do not increase the guess
- Instead, **subtract a lower-order term** from it:
  - $c\,n - d$ instead of $c\,n$;
  - $c\,n^2 - d\,n$ instead of $c\,n^2$
- The stronger hypothesis makes the induction work

Avoid pitfalls

- In the naive attempt it would be wrong to conclude

$$\text{“}T(n) \leq c\,n + 1 = O(n)\text{, done”}$$

- The induction must prove *exactly* the form of the inductive hypothesis, $T(n) \leq c\,n$, with the *same* constant c

- Otherwise the constant would grow at each step of the recursion

Changing variables

Sometimes an algebraic manipulation,
such as a change of variables,
transforms an unknown recurrence into a known one,
whose solution can then be reused.

Example: $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log_2 n$ (1/3)

Consider

$$T(n) = 2\,T\left(\lfloor\sqrt{n}\rfloor\right) + \log_2 n,$$

with boundary condition $T(2) = 1$.

Step 1 (rename).

- We ignore the rounding, that is, we assume that $n = 2^{2^k}$, so that all the square roots are exact powers of 2
- We rename $m = \log_2 n$, that is, $n = 2^m$ and $\sqrt{n} = 2^{m/2}$:

$$T(2^m) = 2\,T\left(2^{m/2}\right) + m$$

Example: $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log_2 n$ (2/3)

Step 2 (new function).

- We define $R(m) = T(2^m)$ and obtain

$$R(m) = 2\,R\left(\frac{m}{2}\right) + m,$$

  which is the recurrence $2T(n/2) + n$ solved with the expansion method

- Since $T(2) = 1$, we have $R(1) = 1$, and therefore

$$R(m) = m \log_2 m + m$$

Example: $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \log_2 n$ (3/3)

Step 3 (back to T).

- With $m = \log_2 n$:

$$\begin{aligned} T(n) &= R(\log_2 n) \\ &= \log_2 n \cdot \log_2 \log_2 n + \log_2 n \\ &= \Theta(\log_2 n \cdot \log_2 \log_2 n) \end{aligned}$$

Check:

- $T(4) = 2\,T(2) + 2 = 4$ and $R(2) = 2 \cdot 1 + 2 = 4$
- $T(16) = 2\,T(4) + 4 = 12$ and $R(4) = 4 \cdot 2 + 4 = 12$

Summary (1/2)

The substitution method:

- guess the form of the solution, then prove it by induction
- prove O and Ω separately; together they give Θ
- the freedom of choosing n_0 handles the base cases

The recurrence $2T(\lfloor n/2 \rfloor) + n$:

- $T(n) \leq 2\, n \log_2 n$ for $n \geq 2$, with base cases $n = 2, 3$
- $T(n) \geq \frac{1}{4}\, n \log_2 n$ for $n \geq 1$
- hence $T(n) = \Theta(n \log_2 n)$: the floor does not change the order of growth

Summary (2/2)

Strengthening the guess:

- for $2T(\lfloor n/2 \rfloor) + 1$ the guess $c\,n$ fails by a lower-order term
- the guess $c\,n - d$ works: $n \leq T(n) \leq 2n - 1$
- the induction must prove exactly the inductive hypothesis, with the same constant

Changing variables:

- $m = \log_2 n$ turns $2T(\lfloor \sqrt{n} \rfloor) + \log_2 n$ into $2R(m/2) + m$
- solution $\Theta(\log_2 n \cdot \log_2 \log_2 n)$

The recursion-tree method

Introduction: The recursion-tree method

- The substitution method proves a guess, but does not produce it
- The recursion tree is the tool that produces good guesses, and in many cases it directly gives the solution
- We apply it to four recurrences: level costs that decrease, stay constant or increase, and an unbalanced tree

Recursion trees

- In a recursion tree, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations. The children of a node are the subproblems generated by it
- It can be used to solve recurrences, that is, to determine their asymptotic complexity, or to produce a good guess to be verified with the substitution method
- Main idea: we sum the costs within each level of the tree to obtain a set of per-level costs, and then we sum all the per-level costs to determine the total cost of all the levels of the recursion

The recursion-tree method

In the **recursion-tree method** we determine:

1. the size of the subproblems and the number of nodes at each depth i;
2. the cost of each level (number of nodes $\times$ cost of each node);
3. the depth of the tree (when the subproblem size reaches the boundary condition) and the cost of the leaves;
4. the sum of the costs of all levels.

- We apply the method to four recurrences, which show the three possible behaviours of the level costs (decreasing, constant, increasing) and the case of an unbalanced tree

Decreasing level costs: $T(n) = 3T(n/4) + c\,n^2$

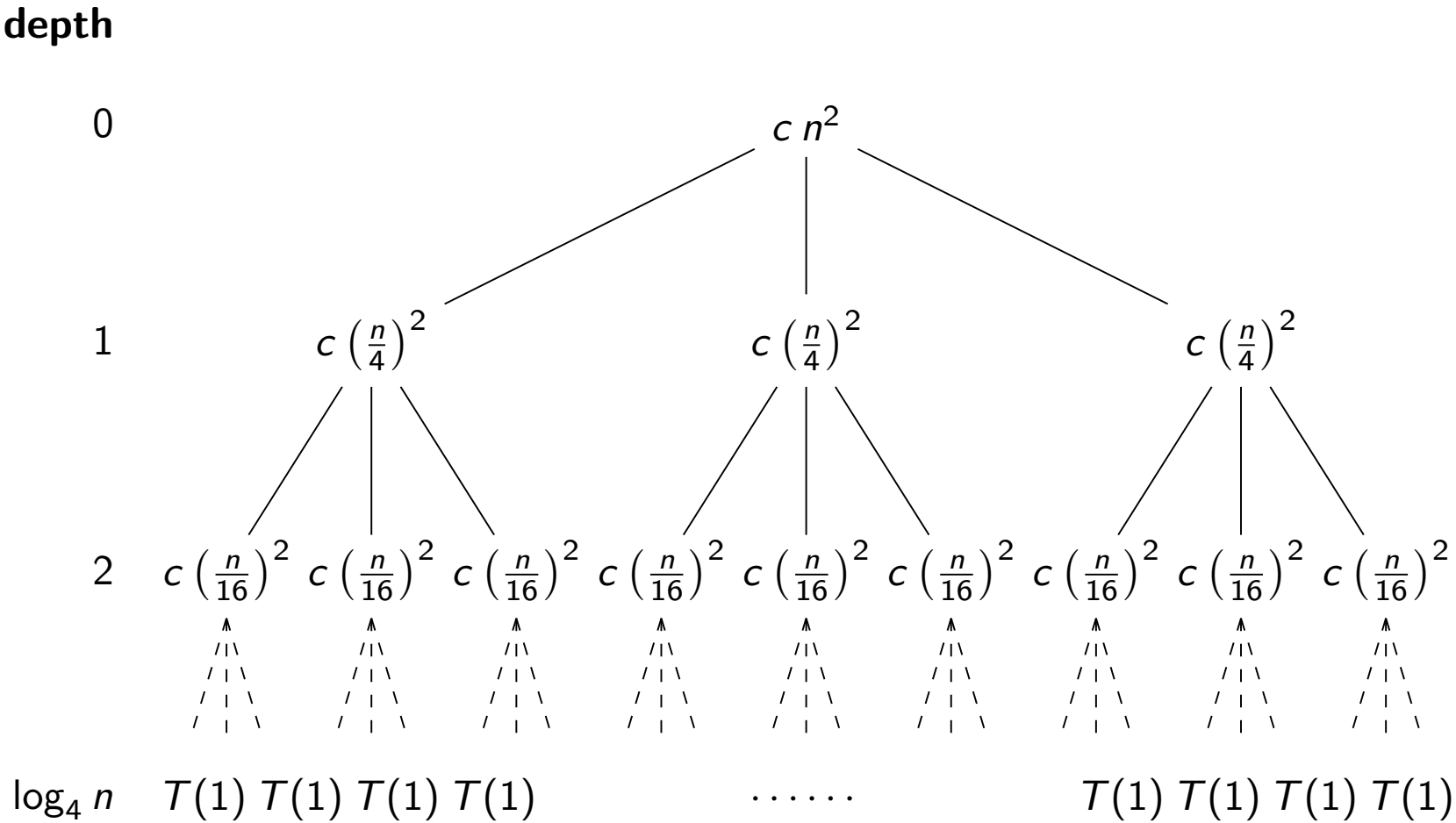
- Consider the recurrence ($c > 0$ a constant)

$$T(n) = 3\,T\left(\frac{n}{4}\right) + c\,n^2$$

- For convenience, we assume that n is an exact power of 4, so that all subproblem sizes are integers

- In the recursion tree every node has three children, the subproblem size is divided by 4 at each level, and the leaves are at depth $\log_4 n$

Recursion tree of $T(n) = 3T(n/4) + c\,n^2$ (1/2)



Recursion tree of $T(n) = 3T(n/4) + c\,n^2$ (2/2)

The cost of each level of the tree:

depth	**cost of the level**
0	$c\,n^2$
1	$\frac{3}{16}\,c\,n^2$
2	$\left(\frac{3}{16}\right)^2 c\,n^2$
$\vdots$	$\vdots$
$\log_4 n$	$\Theta\left(n^{\log_4 3}\right)$

Reading the recursion tree

- The $c\,n^2$ term at the root represents the cost at the top level of the recursion (dividing and combining)
- The three subtrees of the root represent the costs incurred by the subproblems of size $\frac{n}{4}$; at the next depth the size becomes $\frac{n}{16}$, and so on
- The leaves represent the points where the recursion "bottoms out", that is, where the *base case* of the recursion is reached

Example: solving $T(n) = 3T(n/4) + c\,n^2$ (1/6)

Step 1 (sizes and nodes).

- The subproblem size of each node at depth i is $\frac{n}{4^i}$
- Each level has 3 times as many nodes as the previous level, so the number of nodes at depth i is 3^i

Step 2 (cost of a level).

- Each node at depth i ($i = 0, 1, 2, \ldots, \log_4 n - 1$) has a cost of $c\left(\frac{n}{4^i}\right)^2$
- Multiplying, the overall cost of all the nodes at depth i is

$$3^i\, c \left(\frac{n}{4^i}\right)^2 = \left(\frac{3}{16}\right)^i c\, n^2$$

Example: solving $T(n) = 3T(n/4) + c\,n^2$ (2/6)

Step 3 (depth and leaves).

- The subproblem size reaches 1 when $\frac{n}{4^i} = 1$, that is, when $i = \log_4 n$
- Accordingly, the tree has $\log_4 n + 1$ levels (at depths $0, 1, 2, \ldots, \log_4 n$)
- The bottom level has, using $a^{\log_b n} = n^{\log_b a}$,

$$3^{\log_4 n} = n^{\log_4 3}$$

nodes (the leaves of the tree), each contributing a constant cost $T(1)$

- The total cost of the bottom level is

$$n^{\log_4 3}\, T(1) = \Theta\left(n^{\log_4 3}\right)$$

Example: solving $T(n) = 3T(n/4) + c\,n^2$ (3/6)

Step 4 (sum). Summing up the costs of all levels we obtain

$$T(n) = c\,n^2 + \frac{3}{16}\,c\,n^2 + \left(\frac{3}{16}\right)^2 c\,n^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} c\,n^2 + \Theta\left(n^{\log_4 3}\right)$$

$$= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right).$$

Using the geometric series we obtain the exact value

$$T(n) = \frac{16}{13}\left(1 - \left(\frac{3}{16}\right)^{\log_4 n}\right) c\,n^2 + \Theta\left(n^{\log_4 3}\right).$$

Example: solving $T(n) = 3T(n/4) + c\,n^2$ (4/6)

Upper bound.

- It is simpler to bound the finite sum by the infinite decreasing geometric series:

$$\begin{aligned} T(n) &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{1}{1-\frac{3}{16}}\, c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{16}{13}\, c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= O(n^2), \end{aligned}$$

since $\log_4 3 \approx 0.79 < 2$, so that $n^{\log_4 3} = O(n^2)$.

Example: solving $T(n) = 3T(n/4) + c\,n^2$ (5/6)

Lower bound.

- All the terms of the sum are non-negative, and the root alone costs $c\,n^2$
- Hence

$$T(n) \geq c\,n^2 = \Omega(n^2)$$

Example: solving $T(n) = 3T(n/4) + c\,n^2$ (6/6)

Numerical check ($c = 1,\ T(1) = 1$):

- The recurrence gives

$$T(4) = 19, \qquad T(16) = 313, \qquad T(64) = 5035,$$

the values of the exact formula above

- The ratios are

$$\frac{T(n)}{n^2} = 1.1875,\ 1.2227,\ 1.2292,$$

which approach $\frac{16}{13} \approx 1.2308$

Decreasing level costs: the root dominates

- The solution of $T(n) = 3T(n/4) + c\,n^2$ is

$$T(n) = \Theta(n^2)$$

- The costs of the levels decrease geometrically
- The total cost is dominated by the cost of the **root**

Constant level costs: $T(n) = 2T(n/2) + c\,n$

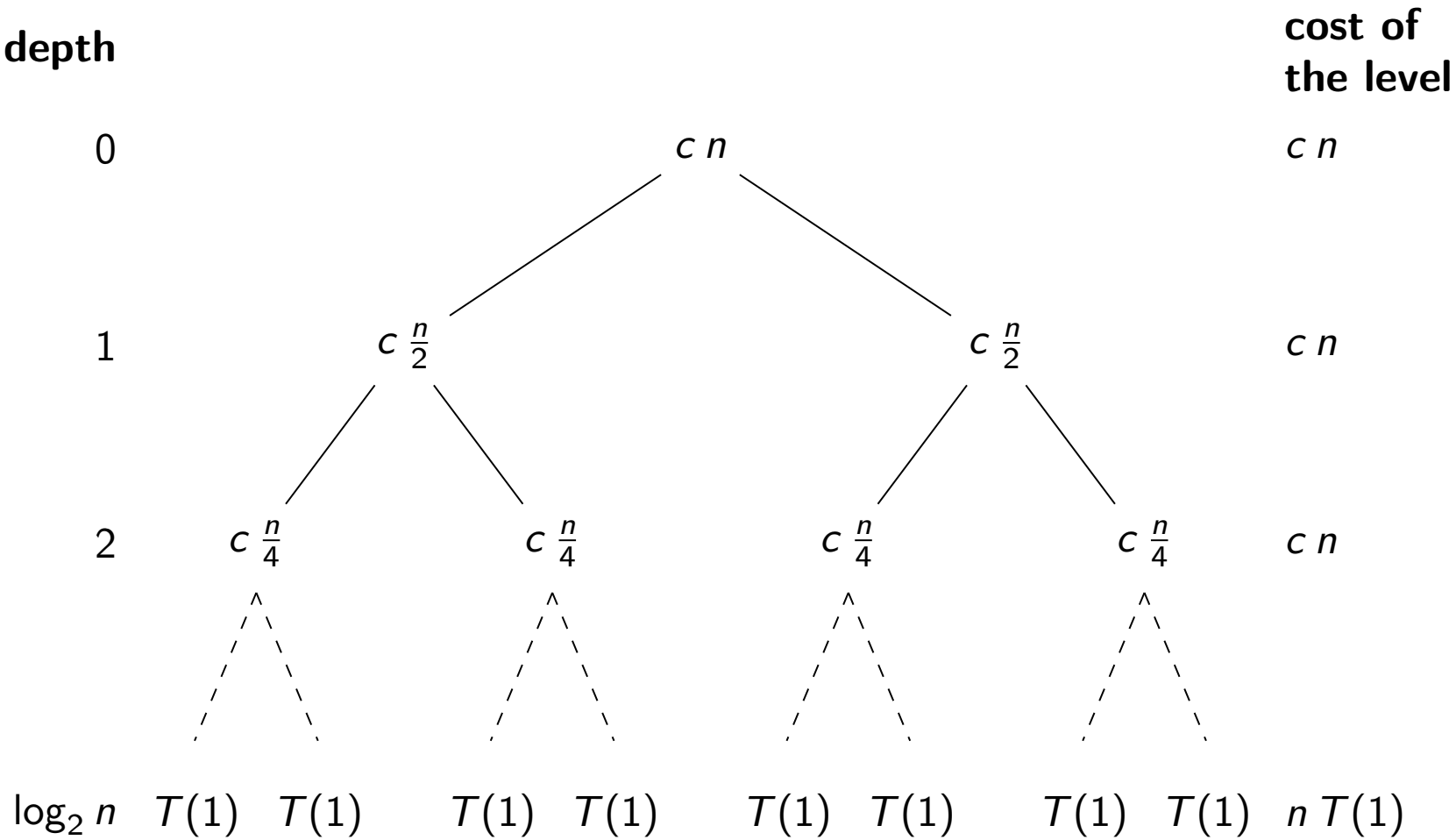
- Consider the recurrence ($c > 0$ a constant), which describes algorithms such as `MergeSort`:

$$T(n) = 2\,T\left(\frac{n}{2}\right) + c\,n,$$

  where n is an exact power of 2

- In its recursion tree every level has total cost $c\,n$

Recursion tree of $T(n) = 2T(n/2) + c\,n$



Example: solving $T(n) = 2T(n/2) + c\,n$ (1/2)

Steps 1 and 2.

- At depth i there are 2^i nodes, each corresponding to a subproblem of size $\frac{n}{2^i}$ and with cost $c\,\frac{n}{2^i}$
- The cost of each level is therefore

$$2^i\, c\, \frac{n}{2^i} = c\, n$$

Step 3.

- The subproblem size reaches 1 at depth $\log_2 n$
- The bottom level has $2^{\log_2 n} = n$ leaves, with total cost $n\,T(1) = \Theta(n)$

Example: solving $T(n) = 2T(n/2) + c\,n\ (2/2)$

Step 4. Summing the costs of all levels:

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_2 n - 1} c\,n + n\,T(1) \\ &= c\,n \log_2 n + n\,T(1) \\ &= \Theta(n \log_2 n) \end{aligned}$$

- For $c = 1$ and $T(1) = 1$ we obtain again $T(n) = n \log_2 n + n$, as with the expansion method

Constant level costs: all levels cost the same

- The solution of $T(n) = 2T(n/2) + c\,n$ is

$$T(n) = \Theta(n \log_2 n)$$

- All the levels have the **same cost** $c\,n$
- There are $\Theta(\log_2 n)$ levels

An unbalanced tree: $T(n) = T(n/3) + T(2n/3) + c\,n$

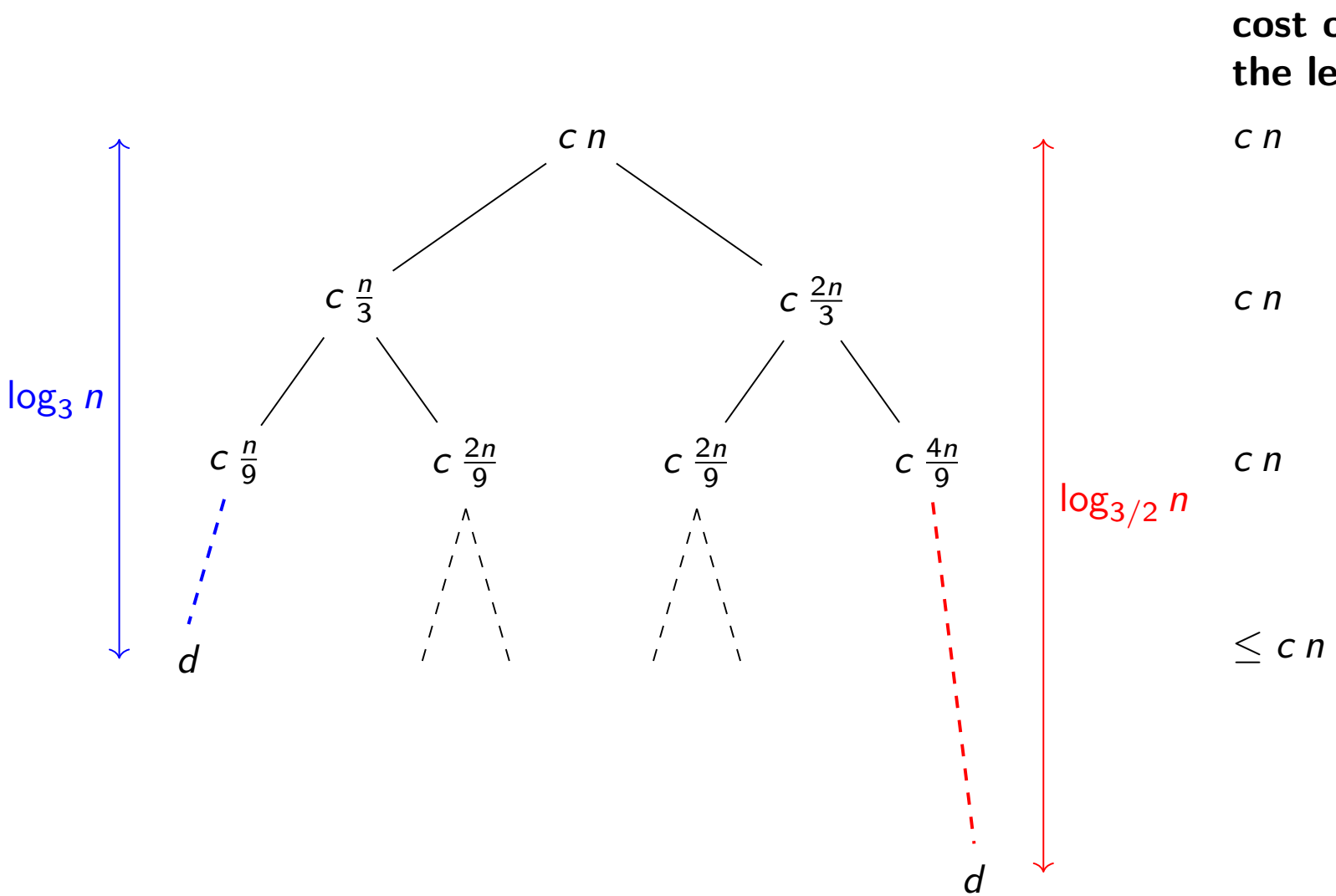
- Consider the recurrence ($c, d > 0$ constants), defined for every real number $n > 0$:

$$T(n) = \begin{cases} d & \text{if } 0 < n < 1 \\ T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + c\,n & \text{if } n \geq 1 \end{cases}$$

- Working with real sizes allows us to avoid floors and ceilings

- In the recursion tree the shortest root-to-leaf path (blue) has length about $\log_3 n$, the longest (red) about $\log_{3/2} n$

Recursion tree of $T(n) = T(n/3) + T(2n/3) + c\,n$



The shape of the unbalanced tree (1/2)

- Each internal node of size s has two children of sizes $\frac{s}{3}$ and $\frac{2s}{3}$, whose sum is s

  Hence the sizes of the nodes at any given depth sum to at most n (exactly n if the level is complete)

- The **shortest path** from the root to a leaf is the leftmost one:

$$n \to \frac{n}{3} \to \frac{n}{9} \to \cdots$$

  A node at depth i on this path has size $\frac{n}{3^i}$, which is ≥ 1 as long as $i \leq \log_3 n$

The shape of the unbalanced tree (2/2)

- The **longest path** is the rightmost one:

$$n \to \frac{2n}{3} \to \frac{4n}{9} \to \cdots$$

  A node at depth i on this path has size $\left(\frac{2}{3}\right)^i n$, which is ≥ 1 as long as $i \leq \log_{3/2} n$

- Unlike the previous examples, the tree is **not complete**: the leaves are at different depths, between $\log_3 n$ and $\log_{3/2} n$ (approximately)

Example: bounds for the unbalanced tree (1/4)

Upper bound: internal nodes.

- Every node at depth i has size at most $\left(\frac{2}{3}\right)^i n$
- An internal node has size ≥ 1, so internal nodes appear only at depths $i \leq \log_{3/2} n$
- At each depth, the internal nodes have total size at most n, so their total cost is at most $c\,n$
- Hence the cost of the internal nodes is at most

$$c\,n\left(\log_{3/2} n + 1\right)$$

Example: bounds for the unbalanced tree (2/4)

Upper bound: leaves.

- A leaf has size < 1, but its parent has size ≥ 1, so a leaf has size $\geq \frac{1}{3}$
- The sizes of all the leaves sum to exactly n (each internal node splits its size among its children)
- Hence there are at most $3n$ leaves, with total cost at most $3\,d\,n = O(n)$

Therefore

$$T(n) \leq c\,n\left(\log_{3/2} n + 1\right) + 3\,d\,n = O(n \log_2 n),$$

since $\log_{3/2} n = \frac{\log_2 n}{\log_2(3/2)}$ differs from $\log_2 n$ by a constant factor.

Example: bounds for the unbalanced tree (3/4)

Lower bound.

- Every node at depth i has size at least $\frac{n}{3^i}$
- Hence, for every depth $i \leq \log_3 n$, *all* the nodes at depth i have size ≥ 1: they are internal nodes, the level is complete (2^i nodes) and their sizes sum to exactly n
- The cost of each of these levels is exactly $c\,n$
- There are $\lfloor \log_3 n \rfloor + 1 \geq \log_3 n$ such levels (depths $0, 1, \ldots, \lfloor \log_3 n \rfloor$), so

$$T(n) \geq c\,n \log_3 n = \Omega(n \log_2 n)$$

Example: bounds for the unbalanced tree (4/4)

Numerical check ($c = d = 1$): computing $T(n)$ exactly from the recurrence,

n	$n \log_3 n$	$T(n)$	$n(\log_{3/2} n + 1) + 3n$
100	419.2	≈ 929.2	1 535.8
1000	6 287.7	$\approx 13\,002.3$	21 036.6

- Both values of $T(n)$ lie between the lower bound $n \log_3 n$ and the upper bound $n(\log_{3/2} n + 1) + 3n$

The unbalanced tree: longest and shortest path

- The solution of $T(n) = T(n/3) + T(2n/3) + c\,n$ is

$$T(n) = \Theta(n \log_2 n)$$

- Each level costs at most $c\,n$
- The upper bound uses the **longest path** ($\log_{3/2} n$) and the lower bound uses the **shortest path** ($\log_3 n$)
- Both are $\Theta(\log_2 n)$

Remark: the tree is far from complete

- If the tree were a complete binary tree of height $\log_{3/2} n$, it would have

$$2^{\log_{3/2} n} = n^{\log_{3/2} 2} \approx n^{1.71}$$

  leaves, which is $\omega(n \log_2 n)$

- The argument above shows that the tree is far from complete: the actual number of leaves is $O(n)$

- When the leaves are not easy to count, a safe approach is to use the recursion tree only to *guess* the solution and then to prove it with the substitution method

Example: solving $T(n) = 4T(n/2) + n$ (1/3)

Consider the recurrence

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 4\,T\left(\frac{n}{2}\right) + n & \text{if } n > 1 \end{cases}$$

where n is an exact power of 2.

Steps 1 and 2.

- At depth i there are 4^i nodes, each of size $\frac{n}{2^i}$ and cost $\frac{n}{2^i}$
- The cost of the level is

$$4^i \frac{n}{2^i} = 2^i\, n$$

Example: solving $T(n) = 4T(n/2) + n$ (2/3)

Step 3. The size reaches 1 at depth $\log_2 n$, where there are $4^{\log_2 n} = n^2$ leaves:

depth	number of nodes	cost of each node	cost of the level
0	1	n	n
1	4	$n/2$	$2n$
2	16	$n/4$	$4n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	4^i	$n/2^i$	$2^i n$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	$4^{\log_2 n} = n^2$	$T(1) = 1$	n^2

Example: solving $T(n) = 4T(n/2) + n$ (3/3)

Step 4. The costs *increase* geometrically going down the tree. Using $\sum_{i=0}^{k-1} 2^i = 2^k - 1$,

$$T(n) = \sum_{i=0}^{\log_2 n - 1} 2^i\, n + n^2 = n\left(2^{\log_2 n} - 1\right) + n^2$$

$$= n(n-1) + n^2 = 2n^2 - n.$$

- Hence $T(n) = 2n^2 - n = \Theta(n^2)$: indeed $\lim_{n\to\infty} \frac{2n^2-n}{n^2} = 2 \in \mathbb{R}_{>0}$
- Check: $T(2) = 4 + 2 = 6$, $T(4) = 24 + 4 = 28$, $T(8) = 112 + 8 = 120 = 2 \cdot 64 - 8$

Increasing level costs: the leaves dominate

- The solution of $T(n) = 4T(n/2) + n$ is

$$T(n) = \Theta(n^2)$$

- The costs of the levels increase geometrically
- The total cost is dominated by the **leaves**, whose number is

$$4^{\log_2 n} = n^{\log_2 4} = n^2$$

Summary (1/2)

The recursion-tree method:

- each node is the cost of one subproblem; sum the costs level by level
- sizes and nodes at depth i, cost of a level, depth and leaves, sum
- it solves the recurrence or produces a guess for the substitution method

Balanced trees:

- $3T(n/4) + c\,n^2$: decreasing level costs, the root dominates, $\Theta(n^2)$
- $2T(n/2) + c\,n$: constant level costs, $\Theta(n \log_2 n)$
- $4T(n/2) + n$: increasing level costs, the leaves dominate, $\Theta(n^2)$

Summary (2/2)

An unbalanced tree:

- $T(n/3) + T(2n/3) + c\,n$: every level costs at most $c\,n$
- upper bound with the longest path $\log_{3/2} n$, lower bound with the shortest path $\log_3 n$
- solution $\Theta(n \log_2 n)$; the number of leaves is $O(n)$

A safe approach:

- when the leaves are not easy to count, guess with the tree and prove with substitution

Towards the Master Theorem

Introduction: Towards the Master Theorem

- The three balanced trees of the previous section are instances of the same picture
- We now draw the recursion tree of a general recurrence in standard form
- The three behaviours that we observe there are the starting point of the Master Theorem, later in the course

A general recurrence in standard form

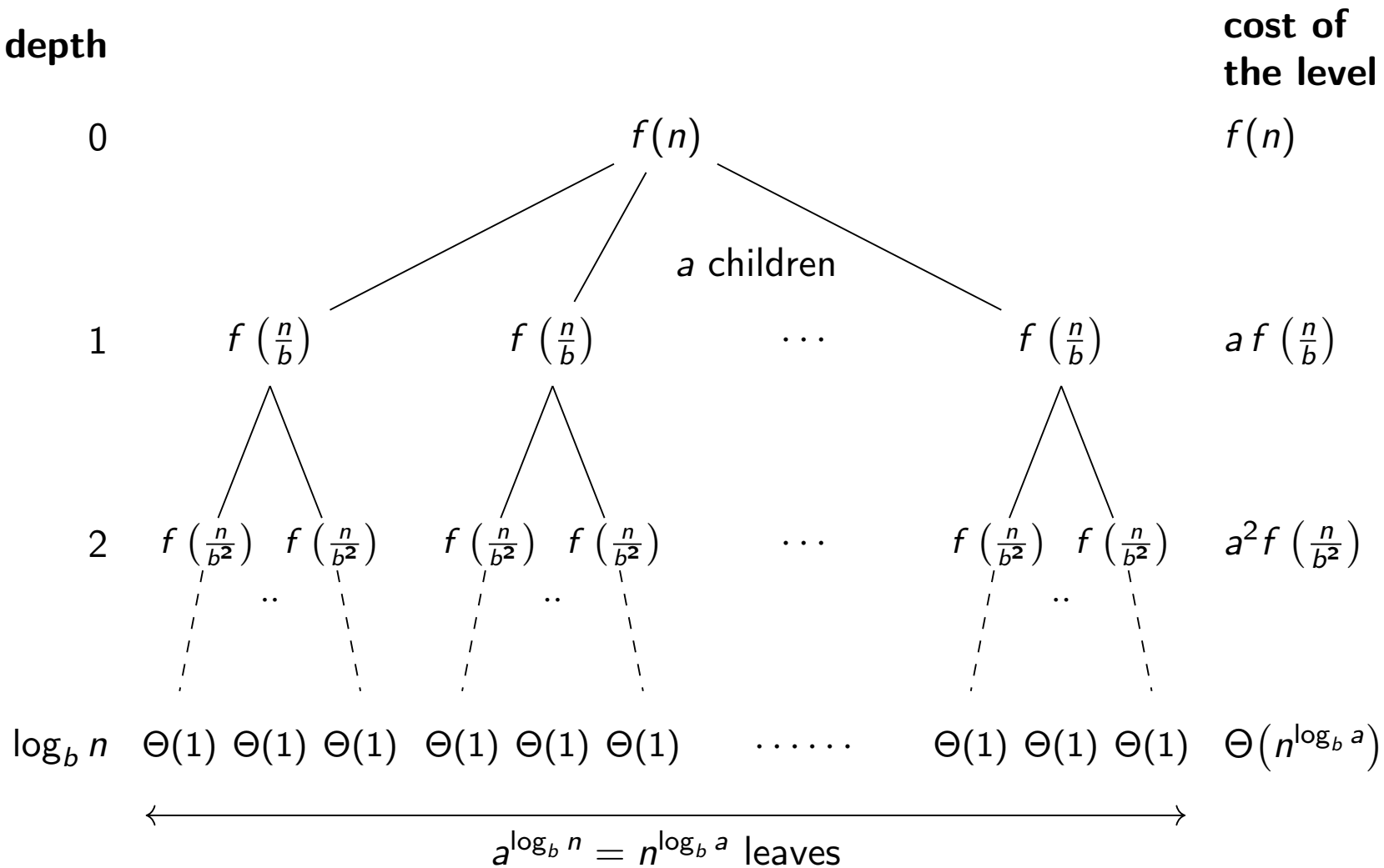
- We consider

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ a\,T\left(\frac{n}{b}\right) + f(n) & \text{if } n > 1 \end{cases}$$

  with $a \geq 1$, $b > 1$ constants and n an exact power of b

- We draw its recursion tree, for n exact power of b

Recursion tree of $T(n) = a\,T(n/b) + f(n)$



Costs in the general recursion tree (1/2)

- At depth j there are a^j nodes, each corresponding to a subproblem of size $\frac{n}{b^j}$ with cost $f\left(\frac{n}{b^j}\right)$
- The cost of the level is

$$a^j f\left(\frac{n}{b^j}\right)$$

- The subproblem size reaches 1 at depth $\log_b n$
- The number of leaves is, using $a^{\log_b n} = n^{\log_b a}$,

$$a^{\log_b n} = n^{\log_b a},$$

and their total cost is $\Theta\left(n^{\log_b a}\right)$

Costs in the general recursion tree (2/2)

- Summing the costs of all levels:

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \tag{1}$$

- The solution depends on how the level costs $a^j f(n/b^j)$ evolve with j

- That is, on the comparison between
  - $f(n)$, the cost of the root, and
  - $n^{\log_b a}$, the cost of the leaves

Three behaviours (1/2)

There are three possible behaviours of (1), all of which we have already met:

1. **The leaves dominate**: the level costs increase geometrically going down the tree, and $T(n) = \Theta\left(n^{\log_b a}\right)$
   Example: $T(n) = 4T(n/2) + n$, with $n^{\log_2 4} = n^2$ leaves, $T(n) = \Theta(n^2)$

2. **All levels have (about) the same cost**: the cost of each of the $\log_b n + 1$ levels is $\Theta\left(n^{\log_b a}\right)$, and $T(n) = \Theta\left(n^{\log_b a} \log_2 n\right)$
   Example: $T(n) = 2T(n/2) + c\,n$, with $n^{\log_2 2} = n$, $T(n) = \Theta(n \log_2 n)$

Three behaviours (2/2)

3. **The root dominates**: the level costs decrease geometrically going down the tree, and $T(n) = \Theta(f(n))$
   Example: $T(n) = 3T(n/4) + c\,n^2$, with $f(n) = c\,n^2$, $T(n) = \Theta(n^2)$

In each case the solution is decided by the comparison between $f(n)$ and $n^{\log_b a}$.

The Master Theorem

- The **Master Theorem**, studied later in the course, makes these three behaviours precise
- It compares $f(n)$ with $n^{\log_b a}$
- It gives the solution of a recurrence in standard form directly, without drawing the recursion tree

Summary

The general recursion tree:

- a^j nodes at depth j, level cost $a^j f(n/b^j)$
- $n^{\log_b a}$ leaves, with total cost $\Theta\left(n^{\log_b a}\right)$

Three behaviours:

- the leaves dominate: $\Theta\left(n^{\log_b a}\right)$
- all levels cost the same: $\Theta\left(n^{\log_b a} \log_2 n\right)$
- the root dominates: $\Theta(f(n))$

Summary: which method when

Introduction: Summary: which method when

- We have seen three methods to solve recurrences, and the Master Theorem will be a fourth
- Each method fits a different kind of recurrence
- We collect here when to use each of them

Which method when (1/2)

- **Expansion method**: recurrences with a *single* recursive term, for example $T(n-1)+f(n)$, $2T(n-1)+1$, $T(n/2)+c$, $2T(n/2)+n$

  It gives the exact closed form when the resulting summation can be computed (arithmetic or geometric series)

- **Substitution method**: when we have a guess and want a *rigorous* proof, in particular with floors and ceilings and explicit boundary conditions

  Prove O and Ω separately; handle the base cases by choosing n_0; if the induction fails by a lower-order term, subtract a lower-order term from the guess. It can be combined with a change of variables

Which method when (2/2)

- **Recursion-tree method**: to visualise the costs and to obtain a guess; ideal when the recursive terms have *different sizes* (unbalanced trees, as $T(n/3) + T(2n/3) + c\,n$)

  Compute the cost per level, the depth, the number of leaves, and sum using the geometric series

- **Master Theorem** (later in the course): recurrences in *standard form* $a\,T(n/b) + f(n)$

  It gives the answer directly by comparing $f(n)$ with $n^{\log_b a}$. It does not apply to $T(n-1) + f(n)$ or to unbalanced splits

Summary: key points of the chapter (1/2)

- A recurrence describes a function $T(n)$ in terms of its values on smaller inputs; the number of steps of a recursive algorithm satisfies one
- Solving a recurrence means finding its closed form or its order of growth; floors, ceilings and boundary conditions can usually be neglected
- The expansion method unrolls the recurrence and sums: $T(n-1)+n=\Theta(n^2)$, $2T(n-1)+1=\Theta(2^n)$, $2T(n/2)+n=\Theta(n\log_2 n)$

Summary: key points of the chapter (2/2)

- The substitution method proves a guess by induction; when it fails by a lower-order term, subtract a lower-order term from the guess
- The recursion-tree method sums the costs level by level; it also handles unbalanced trees such as $T(n/3) + T(2n/3) + c\,n = \Theta(n \log_2 n)$
- In standard form, the solution depends on whether the leaves, all the levels or the root dominate: the Master Theorem makes this precise